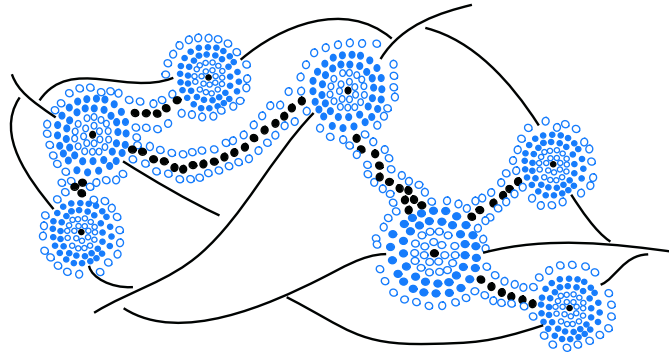


Chapter

3

Fundamental Data Structures



Contents

3.1 Using Arrays	104
3.1.1 Storing Game Entries in an Array	104
3.1.2 Sorting an Array	110
3.1.3 java.util Methods for Arrays and Random Numbers	112
3.1.4 Simple Cryptography with Character Arrays	115
3.1.5 Two-Dimensional Arrays and Positional Games	118
3.2 Singly Linked Lists	122
3.2.1 Implementing a Singly Linked List Class	126
3.3 Circularly Linked Lists	128
3.3.1 Round-Robin Scheduling	128
3.3.2 Designing and Implementing a Circularly Linked List	129
3.4 Doubly Linked Lists	132
3.4.1 Implementing a Doubly Linked List Class	135
3.5 Equivalence Testing	138
3.5.1 Equivalence Testing with Arrays	139
3.5.2 Equivalence Testing with Linked Lists	140
3.6 Cloning Data Structures	141
3.6.1 Cloning Arrays	142
3.6.2 Cloning Linked Lists	144
3.7 Exercises	145

3.1 Using Arrays

In this section, we explore a few applications of arrays—the concrete data structures introduced in Section 1.3 that access their entries using integer indices.

3.1.1 Storing Game Entries in an Array

The first application we study is storing a sequence of high score entries for a video game in an array. This is representative of many applications in which a sequence of objects must be stored. We could just as easily have chosen to store records for patients in a hospital or the names of players on a football team. Nevertheless, let us focus on storing high score entries, which is a simple application that is already rich enough to present some important data-structuring concepts.

To begin, we consider what information to include in an object representing a high score entry. Obviously, one component to include is an integer representing the score itself, which we identify as `score`. Another useful thing to include is the name of the person earning this score, which we identify as `name`. We could go on from here, adding fields representing the date the score was earned or game statistics that led to that score. However, we omit such details to keep our example simple. A Java class, `GameEntry`, representing a game entry, is given in Code Fragment 3.1.

```
1 public class GameEntry {
2     private String name;           // name of the person earning this score
3     private int score;             // the score value
4     /** Constructs a game entry with given parameters.. */
5     public GameEntry(String n, int s) {
6         name = n;
7         score = s;
8     }
9     /** Returns the name field. */
10    public String getName() { return name; }
11    /** Returns the score field. */
12    public int getScore() { return score; }
13    /** Returns a string representation of this entry. */
14    public String toString() {
15        return "(" + name + ", " + score + ")";
16    }
17 }
```

Code Fragment 3.1: Java code for a simple `GameEntry` class. Note that we include methods for returning the name and score for a game entry object, as well as a method for returning a string representation of this entry.

A Class for High Scores

To maintain a sequence of high scores, we develop a class named `Scoreboard`. A scoreboard is limited to a certain number of high scores that can be saved; once that limit is reached, a new score only qualifies for the scoreboard if it is strictly higher than the lowest “high score” on the board. The length of the desired scoreboard may depend on the game, perhaps 10, 50, or 500. Since that limit may vary, we allow it to be specified as a parameter to our `Scoreboard` constructor.

Internally, we will use an array named `board` to manage the `GameEntry` instances that represent the high scores. The array is allocated with the specified maximum capacity, but all entries are initially **null**. As entries are added, we will maintain them from highest to lowest score, starting at index 0 of the array. We illustrate a typical state of the data structure in Figure 3.1, and give Java code to construct such a data structure in Code Fragment 3.2.

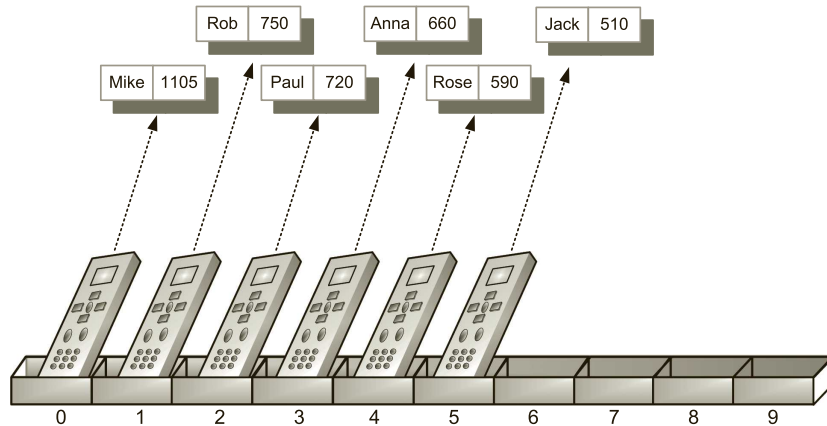


Figure 3.1: An illustration of an array of length ten storing references to six `GameEntry` objects in the cells with indices 0 to 5; the rest are **null** references.

```

1  /** Class for storing high scores in an array in nondecreasing order. */
2  public class Scoreboard {
3      private int numEntries = 0;           // number of actual entries
4      private GameEntry[] board;           // array of game entries (names & scores)
5      /** Constructs an empty scoreboard with the given capacity for storing entries. */
6      public Scoreboard(int capacity) {
7          board = new GameEntry[capacity];
8      }
9      ... // more methods will go here
36 }

```

Code Fragment 3.2: The beginning of a `Scoreboard` class for maintaining a set of scores as `GameEntry` objects. (Completed in Code Fragments 3.3 and 3.4.)

Adding an Entry

One of the most common updates we might want to make to a Scoreboard is to add a new entry. Keep in mind that not every entry will necessarily qualify as a high score. If the board is not yet full, any new entry will be retained. Once the board is full, a new entry is only retained if it is strictly better than one of the other scores, in particular, the last entry of the scoreboard, which is the lowest of the high scores.

Code Fragment 3.3 provides an implementation of an update method for the Scoreboard class that considers the addition of a new game entry.

```

 9  /** Attempt to add a new score to the collection (if it is high enough) */
10  public void add(GameEntry e) {
11      int newScore = e.getScore();
12      // is the new entry e really a high score?
13      if (numEntries < board.length || newScore > board[numEntries-1].getScore()) {
14          if (numEntries < board.length)           // no score drops from the board
15              numEntries++;                       // so overall number increases
16          // shift any lower scores rightward to make room for the new entry
17          int j = numEntries - 1;
18          while (j > 0 && board[j-1].getScore() < newScore) {
19              board[j] = board[j-1];             // shift entry from j-1 to j
20              j--;                               // and decrement j
21          }
22          board[j] = e;                           // when done, add new entry
23      }
24  }
```

Code Fragment 3.3: Java code for inserting a GameEntry object into a Scoreboard.

When a new score is considered, the first goal is to determine whether it qualifies as a high score. This will be the case (see line 13) if the scoreboard is below its capacity, or if the new score is strictly higher than the lowest score on the board.

Once it has been determined that a new entry should be kept, there are two remaining tasks: (1) properly update the number of entries, and (2) place the new entry in the appropriate location, shifting entries with inferior scores as needed.

The first of these tasks is easily handled at lines 14 and 15, as the total number of entries can only be increased if the board is not yet at full capacity. (When full, the addition of a new entry will be counteracted by the removal of the entry with lowest score.)

The placement of the new entry is implemented by lines 17–22. Index *j* is initially set to `numEntries - 1`, which is the index at which the last GameEntry will reside after completing the operation. Either *j* is the correct index for the newest entry, or one or more immediately before it will have lesser scores. The while loop checks the compound condition, shifting entries rightward and decrementing *j*, as long as there is another entry at index *j* - 1 with a score less than the new score.

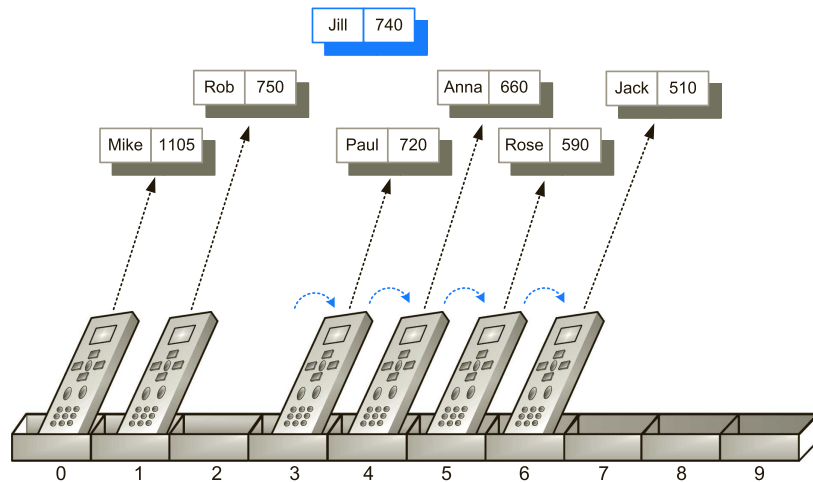


Figure 3.2: Preparing to add Jill's GameEntry object to the board array. In order to make room for the new reference, we have to shift any references to game entries with smaller scores than the new one to the right by one cell.

Figure 3.2 shows an example of the process, just after the shifting of existing entries, but before adding the new entry. When the loop completes, `j` will be the correct index for the new entry. Figure 3.3 shows the result of a complete operation, after the assignment of `board[j] = e`, accomplished by line 22 of the code.

In Exercise C-3.19, we explore how game entry addition might be simplified for the case when we don't need to preserve relative orders.

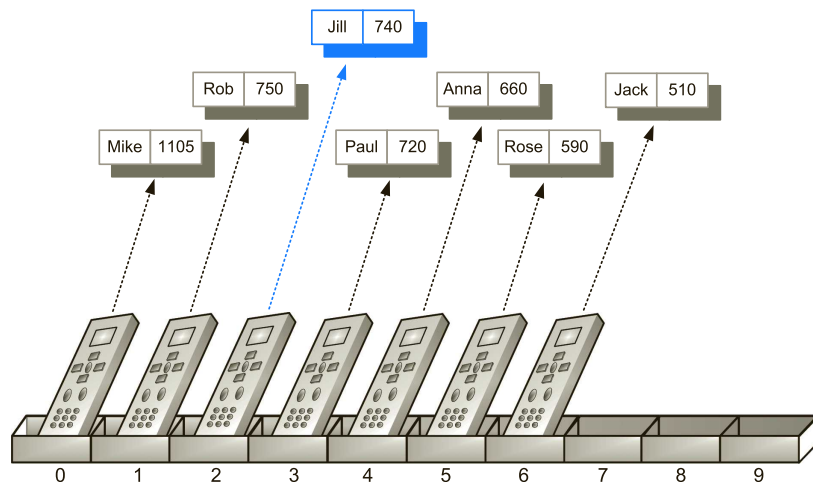


Figure 3.3: Adding a reference to Jill's GameEntry object to the board array. The reference can now be inserted at index 2, since we have shifted all references to GameEntry objects with scores less than the new one to the right.

Removing an Entry

Suppose some hot shot plays our video game and gets his or her name on our high score list, but we later learn that cheating occurred. In this case, we might want to have a method that lets us remove a game entry from the list of high scores. Therefore, let us consider how we might remove a reference to a `GameEntry` object from a `Scoreboard`.

We choose to add a method to the `Scoreboard` class, with signature `remove(i)`, where *i* designates the current index of the entry that should be removed and returned. When a score is removed, any lower scores will be shifted upward, to fill in for the removed entry. If index *i* is outside the range of current entries, the method will throw an `IndexOutOfBoundsException`.

Our implementation for `remove` will involve a loop for shifting entries, much like our algorithm for addition, but in reverse. To remove the reference to the object at index *i*, we start at index *i* and move all the references at indices higher than *i* one cell to the left. (See Figure 3.4.)

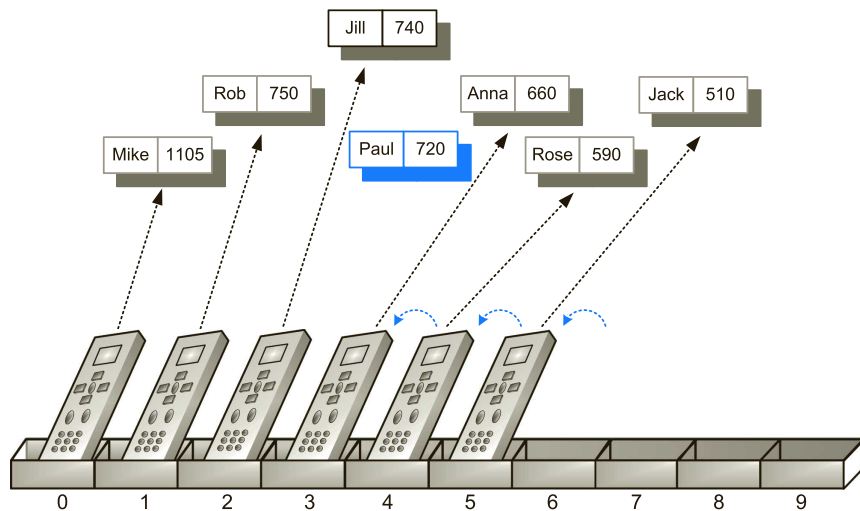


Figure 3.4: An illustration of the removal of Paul's score from index 3 of an array storing references to `GameEntry` objects.

Our implementation of the `remove` method for the `Scoreboard` class is given in Code Fragment 3.4. The details for doing the remove operation contain a few subtle points. The first is that, in order to remove and return the game entry (let's call it *e*) at index *i* in our array, we must first save *e* in a temporary variable. We will use this variable to return *e* when we are done removing it.

```

25  /** Remove and return the high score at index i. */
26  public GameEntry remove(int i) throws IndexOutOfBoundsException {
27      if (i < 0 || i >= numEntries)
28          throw new IndexOutOfBoundsException("Invalid index: " + i);
29      GameEntry temp = board[i];           // save the object to be removed
30      for (int j = i; j < numEntries - 1; j++) // count up from i (not down)
31          board[j] = board[j+1];           // move one cell to the left
32      board[numEntries - 1] = null;         // null out the old last score
33      numEntries--;
34      return temp;                         // return the removed object
35  }

```

Code Fragment 3.4: Java code for performing the `Scoreboard.remove` operation.

The second subtle point is that, in moving references higher than *i* one cell to the left, we don't go all the way to the end of the array. First, we base our loop on the number of current entries, not the capacity of the array, because there is no reason for "shifting" a series of **null** references that may be at the end of the array. We also carefully define the loop condition, `j < numEntries - 1`, so that the last iteration of the loop assigns `board[numEntries-2] = board[numEntries-1]`. There is no entry to shift into cell `board[numEntries-1]`, so we return that cell to **null** just after the loop. We conclude by returning a reference to the removed entry (which no longer has any reference pointing to it within the board array).

Conclusions

In the version of the `Scoreboard` class that is available online, we include an implementation of the `toString()` method, which allows us to display the contents of the current scoreboard, separated by commas. We also include a main method that performs a basic test of the class.

The methods for adding and removing objects in an array of high scores are simple. Nevertheless, they form the basis of techniques that are used repeatedly to build more sophisticated data structures. These other structures may be more general than the array structure above, of course, and often they will have a lot more operations that they can perform than just add and remove. But studying the concrete array data structure, as we are doing now, is a great starting point to understanding these other structures, since every data structure has to be implemented using concrete means.

In fact, later in this book, we will study a Java collections class, `ArrayList`, which is more general than the array structure we are studying here. The `ArrayList` has methods to operate on an underlying array; yet it also eliminates the error that occurs when adding an object to a full array by automatically copying the objects into a larger array when necessary. We will discuss the `ArrayList` class in far more detail in Section 7.2.

3.1.2 Sorting an Array

In the previous subsection, we considered an application for which we added an object to an array at a given position while shifting other elements so as to keep the previous order intact. In this section, we use a similar technique to solve the *sorting* problem, that is, starting with an unordered array of elements and rearranging them into nondecreasing order.

The Insertion-Sort Algorithm

We study several sorting algorithms in this book, most of which are described in Chapter 12. As a warm-up, in this section we describe a simple sorting algorithm known as *insertion-sort*. The algorithm proceeds by considering one element at a time, placing the element in the correct order relative to those before it. We start with the first element in the array, which is trivially sorted by itself. When considering the next element in the array, if it is smaller than the first, we swap them. Next we consider the third element in the array, swapping it leftward until it is in its proper order relative to the first two elements. We continue in this manner with the fourth element, the fifth, and so on, until the whole array is sorted. We can express the insertion-sort algorithm in pseudocode, as shown in Code Fragment 3.5.

Algorithm InsertionSort(A):

Input: An array A of n comparable elements

Output: The array A with elements rearranged in nondecreasing order

for k from 1 to $n - 1$ **do**

 Insert $A[k]$ at its proper location within $A[0], A[1], \dots, A[k]$.

Code Fragment 3.5: High-level description of the insertion-sort algorithm.

This is a simple, high-level description of insertion-sort. If we look back to Code Fragment 3.3 in Section 3.1.1, we see that the task of inserting a new entry into the list of high scores is almost identical to the task of inserting a newly considered element in insertion-sort (except that game scores were ordered from high to low). We provide a Java implementation of insertion-sort in Code Fragment 3.6, using an outer loop to consider each element in turn, and an inner loop that moves a newly considered element to its proper location relative to the (sorted) subarray of elements that are to its left. We illustrate an example run of the insertion-sort algorithm in Figure 3.5.

We note that if an array is already sorted, the inner loop of insertion-sort does only one comparison, determines that there is no swap needed, and returns back to the outer loop. Of course, we might have to do a lot more work than this if the input array is extremely out of order. In fact, we will have to do the most work if the input array is in decreasing order.

```

1  /** Insertion-sort of an array of characters into nondecreasing order */
2  public static void insertionSort(char[] data) {
3      int n = data.length;
4      for (int k = 1; k < n; k++) {           // begin with second character
5          char cur = data[k];                // time to insert cur=data[k]
6          int j = k;                          // find correct index j for cur
7          while (j > 0 && data[j-1] > cur) {  // thus, data[j-1] must go after cur
8              data[j] = data[j-1];          // slide data[j-1] rightward
9              j--;                           // and consider previous j for cur
10         }
11         data[j] = cur;                      // this is the proper place for cur
12     }
13 }

```

Code Fragment 3.6: Java code for performing insertion-sort on a character array.

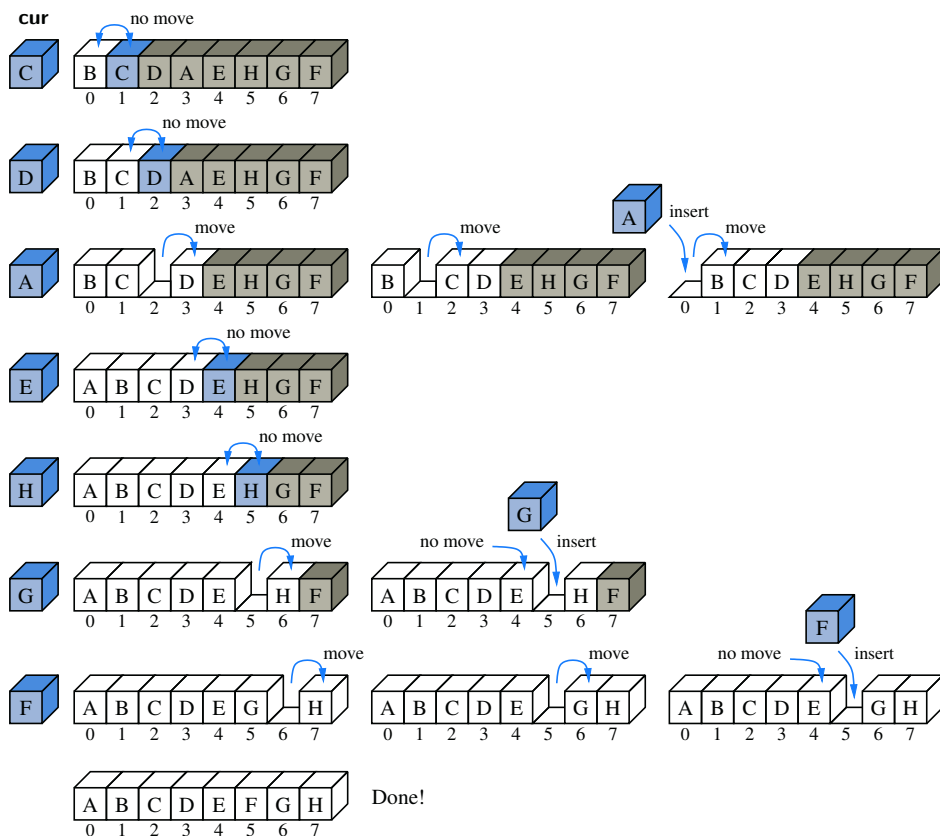


Figure 3.5: Execution of the insertion-sort algorithm on an array of eight characters. Each row corresponds to an iteration of the outer loop, and each copy of the sequence in a row corresponds to an iteration of the inner loop. The current element that is being inserted is highlighted in the array, and shown as the cur value.

3.1.3 java.util Methods for Arrays and Random Numbers

Because arrays are so important, Java provides a class, `java.util.Arrays`, with a number of built-in static methods for performing common tasks on arrays. Later in this book, we will describe the algorithms that several of these methods are based upon. For now, we provide an overview of the most commonly used methods of that class, as follows (more discussion is in Section 3.5.1):

- `equals(A, B)`: Returns true if and only if the array *A* and the array *B* are equal. Two arrays are considered equal if they have the same number of elements and every corresponding pair of elements in the two arrays are equal. That is, *A* and *B* have the same values in the same order.
- `fill(A, x)`: Stores value *x* in every cell of array *A*, provided the type of array *A* is defined so that it is allowed to store the value *x*.
- `copyOf(A, n)`: Returns an array of size *n* such that the first *k* elements of this array are copied from *A*, where $k = \min\{n, A.length\}$. If $n > A.length$, then the last $n - A.length$ elements in this array will be padded with default values, e.g., 0 for an array of **int** and **null** for an array of objects.
- `copyOfRange(A, s, t)`: Returns an array of size $t - s$ such that the elements of this array are copied in order from $A[s]$ to $A[t - 1]$, where $s < t$, padded as with `copyOf()` if $t > A.length$.
- `toString(A)`: Returns a String representation of the array *A*, beginning with `[`, ending with `]`, and with elements of *A* displayed separated by string `,` . The string representation of an element $A[i]$ is obtained using `String.valueOf(A[i])`, which returns the string `"null"` for a **null** reference and otherwise calls $A[i].toString()$.
- `sort(A)`: Sorts the array *A* based on a natural ordering of its elements, which must be comparable. Sorting algorithms are the focus of Chapter 12.
- `binarySearch(A, x)`: Searches the *sorted* array *A* for value *x*, returning the index where it is found, or else the index of where it could be inserted while maintaining the sorted order. The binary-search algorithm is described in Section 5.1.3.

As static methods, these are invoked directly on the `java.util.Arrays` class, not on a particular instance of the class. For example, if *data* were an array, we could sort it with syntax, `java.util.Arrays.sort(data)`, or with the shorter syntax `Arrays.sort(data)` if we first import the `Arrays` class (see Section 1.8).

PseudoRandom Number Generation

Another feature built into Java, which is often useful when testing programs dealing with arrays, is the ability to generate pseudorandom numbers, that is, numbers that appear to be random (but are not necessarily truly random). In particular, Java has a built-in class, `java.util.Random`, whose instances are *pseudorandom number generators*, that is, objects that compute a sequence of numbers that are statistically random. These sequences are not actually random, however, in that it is possible to predict the next number in the sequence given the past list of numbers. Indeed, a popular pseudorandom number generator is to generate the next number, `next`, from the current number, `cur`, according to the formula (in Java syntax):

$$\text{next} = (a * \text{cur} + b) \% n;$$

where `a`, `b`, and `n` are appropriately chosen integers, and `%` is the modulus operator. Something along these lines is, in fact, the method used by `java.util.Random` objects, with $n = 2^{48}$. It turns out that such a sequence can be proven to be statistically uniform, which is usually good enough for most applications requiring random numbers, such as games. For applications, such as computer security settings, where unpredictable random sequences are needed, this kind of formula should not be used. Instead, ideally a sample from a source that is actually random should be used, such as radio static coming from outer space.

Since the next number in a pseudorandom generator is determined by the previous number(s), such a generator always needs a place to start, which is called its *seed*. The sequence of numbers generated for a given seed will always be the same. The seed for an instance of the `java.util.Random` class can be set in its constructor or with its `setSeed()` method.

One common trick to get a different sequence each time a program is run is to use a seed that will be different for each run. For example, we could use some timed input from a user or we could set the seed to the current time in milliseconds since January 1, 1970 (provided by method `System.currentTimeMillis()`).

Methods of the `java.util.Random` class include the following:

- `nextBoolean()`: Returns the next pseudorandom **boolean** value.
- `nextDouble()`: Returns the next pseudorandom **double** value, between 0.0 and 1.0.
- `nextInt()`: Returns the next pseudorandom **int** value.
- `nextInt(n)`: Returns the next pseudorandom **int** value in the range from 0 up to but not including *n*.
- `setSeed(s)`: Sets the seed of this pseudorandom number generator to the **long** *s*.

An Illustrative Example

We provide a short (but complete) illustrative program in Code Fragment 3.7.

```

1  import java.util.Arrays;
2  import java.util.Random;
3  /** Program showing some array uses. */
4  public class ArrayTest {
5      public static void main(String[ ] args) {
6          int data[ ] = new int[10];
7          Random rand = new Random();           // a pseudo-random number generator
8          rand.setSeed(System.currentTimeMillis()); // use current time as a seed
9          // fill the data array with pseudo-random numbers from 0 to 99, inclusive
10         for (int i = 0; i < data.length; i++)
11             data[i] = rand.nextInt(100);        // the next pseudo-random number
12         int[ ] orig = Arrays.copyOf(data, data.length); // make a copy of the data array
13         System.out.println("arrays equal before sort: " + Arrays.equals(data, orig));
14         Arrays.sort(data);                     // sorting the data array (orig is unchanged)
15         System.out.println("arrays equal after sort: " + Arrays.equals(data, orig));
16         System.out.println("orig = " + Arrays.toString(orig));
17         System.out.println("data = " + Arrays.toString(data));
18     }
19 }

```

Code Fragment 3.7: A simple test of some built-in methods in java.util.Arrays.

We show a sample output of this program below:

```

arrays equal before sort: true
arrays equal after sort: false
orig = [41, 38, 48, 12, 28, 46, 33, 19, 10, 58]
data = [10, 12, 19, 28, 33, 38, 41, 46, 48, 58]

```

In another run, we got the following output:

```

arrays equal before sort: true
arrays equal after sort: false
orig = [87, 49, 70, 2, 59, 37, 63, 37, 95, 1]
data = [1, 2, 37, 37, 49, 59, 63, 70, 87, 95]

```

By using a pseudorandom number generator to determine program values, we get a different input to our program each time we run it. This feature is, in fact, what makes pseudorandom number generators useful for testing code, particularly when dealing with arrays. Even so, we should not use random test runs as a replacement for reasoning about our code, as we might miss important special cases in test runs. Note, for example, that there is a slight chance that the orig and data arrays will be equal even after data is sorted, namely, if orig is already ordered. The odds of this occurring are less than 1 in 3 million, so it's unlikely to happen during even a few thousand test runs; however, we need to reason that this is possible.

3.1.4 Simple Cryptography with Character Arrays

An important application of character arrays and strings is *cryptography*, which is the science of secret messages. This field involves the process of *encryption*, in which a message, called the *plaintext*, is converted into a scrambled message, called the *ciphertext*. Likewise, cryptography studies corresponding ways of performing *decryption*, turning a ciphertext back into its original plaintext.

Arguably the earliest encryption scheme is the *Caesar cipher*, which is named after Julius Caesar, who used this scheme to protect important military messages. (All of Caesar's messages were written in Latin, of course, which already makes them unreadable for most of us!) The Caesar cipher is a simple way to obscure a message written in a language that forms words with an alphabet.

The Caesar cipher involves replacing each letter in a message with the letter that is a certain number of letters after it in the alphabet. So, in an English message, we might replace each A with D, each B with E, each C with F, and so on, if shifting by three characters. We continue this approach all the way up to W, which is replaced with Z. Then, we let the substitution pattern *wrap around*, so that we replace X with A, Y with B, and Z with C.

Converting Between Strings and Character Arrays

Given that strings are immutable, we cannot directly edit an instance to encrypt it. Instead, our goal will be to generate a new string. A convenient technique for performing string transformations is to create an equivalent array of characters, edit the array, and then reassemble a (new) string based on the array.

Java has support for conversions from strings to character arrays and vice versa. Given a string *S*, we can create a new character array matching *S* by using the method, *S.toCharArray()*. For example, if *s*="bird", the method returns the character array *A*={'b', 'i', 'r', 'd'}. Conversely, there is a form of the String constructor that accepts a character array as a parameter. For example, with character array *A*={'b', 'i', 'r', 'd'}, the syntax **new** String(*A*) produces "bird".

Using Character Arrays as Replacement Codes

If we were to number our letters like array indices, so that A is 0, B is 1, C is 2, then we can represent the replacement rule as a character array, encoder, such that A is mapped to encoder[0], B is mapped to encoder[1], and so on. Then, in order to find a replacement for a character in our Caesar cipher, we need to map the characters from A to Z to the respective numbers from 0 to 25. Fortunately, we can rely on the fact that characters are represented in Unicode by integer code points, and the code points for the uppercase letters of the Latin alphabet are consecutive (for simplicity, we restrict our encryption to uppercase letters).

Java allows us to “subtract” two characters from each other, with an integer result equal to their separation distance in the encoding. Given a variable c that is known to be an uppercase letter, the Java computation, $j = c - 'A'$ produces the desired index j . As a sanity check, if character c is 'A', then $j = 0$. When c is 'B', the difference is 1. In general, the integer j that results from such a calculation can be used as an index into our precomputed encoder array, as illustrated in Figure 3.6.

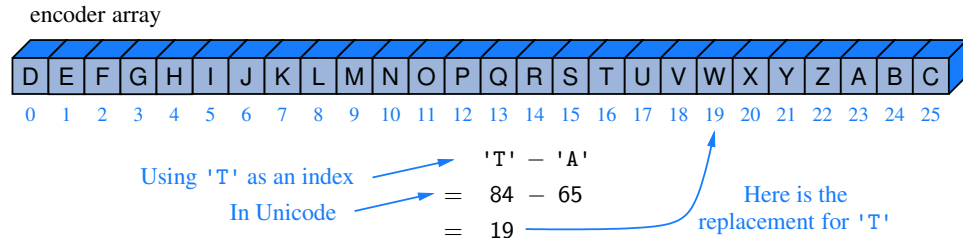


Figure 3.6: Illustrating the use of uppercase characters as indices, in this case to perform the replacement rule for Caesar cipher encryption.

The process of *decrypting* the message can be implemented by simply using a different character array to represent the replacement rule—one that effectively shifts characters in the opposite direction.

In Code Fragment 3.8, we present a Java class that performs the Caesar cipher with an arbitrary rotational shift. The constructor for the class builds the encoder and decoder translation arrays for the given rotation. We rely heavily on modular arithmetic, as a Caesar cipher with a rotation of r encodes the letter having index k with the letter having index $(k + r) \bmod 26$, where $\bmod$ is the *modulo* operator, which returns the remainder after performing an integer division. This operator is denoted with `%` in Java, and it is exactly the operator we need to easily perform the wraparound at the end of the alphabet, for $26 \bmod 26$ is 0, $27 \bmod 26$ is 1, and $28 \bmod 26$ is 2. The decoder array for the Caesar cipher is just the opposite—we replace each letter with the one r places before it, with wraparound; to avoid subtleties involving negative numbers and the modulus operator, we will replace the letter having code k with the letter having code $(k - r + 26) \bmod 26$.

With the encoder and decoder arrays in hand, the encryption and decryption algorithms are essentially the same, and so we perform both by means of a private utility method named `transform`. This method converts a string to a character array, performs the translation diagrammed in Figure 3.6 for any uppercase alphabet symbols, and finally returns a new string, constructed from the updated array.

The main method of the class, as a simple test, produces the following output:

```
Encryption code = DEFGHIJKLMNOPQRSTUVWXYZABC
Decryption code = XYZABCDEFGHIJKLMNOPQRSTUVWXYZ
Secret:  WKH HDJOH LV LQ SODB; PHHW DW MRH'V.
Message: THE EAGLE IS IN PLAY; MEET AT JOE'S.
```

```

1  /** Class for doing encryption and decryption using the Caesar Cipher. */
2  public class CaesarCipher {
3      protected char[] encoder = new char[26];           // Encryption array
4      protected char[] decoder = new char[26];           // Decryption array
5      /** Constructor that initializes the encryption and decryption arrays */
6      public CaesarCipher(int rotation) {
7          for (int k=0; k < 26; k++) {
8              encoder[k] = (char) ('A' + (k + rotation) % 26);
9              decoder[k] = (char) ('A' + (k - rotation + 26) % 26);
10         }
11     }
12     /** Returns String representing encrypted message. */
13     public String encrypt(String message) {
14         return transform(message, encoder);             // use encoder array
15     }
16     /** Returns decrypted message given encrypted secret. */
17     public String decrypt(String secret) {
18         return transform(secret, decoder);              // use decoder array
19     }
20     /** Returns transformation of original String using given code. */
21     private String transform(String original, char[] code) {
22         char[] msg = original.toCharArray();
23         for (int k=0; k < msg.length; k++)
24             if (Character.isUpperCase(msg[k])) {         // we have a letter to change
25                 int j = msg[k] - 'A';                   // will be value from 0 to 25
26                 msg[k] = code[j];                      // replace the character
27             }
28         return new String(msg);
29     }
30     /** Simple main method for testing the Caesar cipher */
31     public static void main(String[] args) {
32         CaesarCipher cipher = new CaesarCipher(3);
33         System.out.println("Encryption code = " + new String(cipher.encoder));
34         System.out.println("Decryption code = " + new String(cipher.decoder));
35         String message = "THE EAGLE IS IN PLAY; MEET AT JOE'S.";
36         String coded = cipher.encrypt(message);
37         System.out.println("Secret: " + coded);
38         String answer = cipher.decrypt(coded);
39         System.out.println("Message: " + answer);        // should be plaintext again
40     }
41 }

```

Code Fragment 3.8: A complete Java class for performing the Caesar cipher.

3.1.5 Two-Dimensional Arrays and Positional Games

Many computer games, be they strategy games, simulation games, or first-person conflict games, involve objects that reside in a two-dimensional space. Software for such *positional games* needs a way of representing objects in a two-dimensional space. A natural way to do this is with a *two-dimensional array*, where we use two indices, say i and j , to refer to the cells in the array. The first index usually refers to a row number and the second to a column number. Given such an array, we can maintain two-dimensional game boards and perform other kinds of computations involving data stored in rows and columns.

Arrays in Java are one-dimensional; we use a single index to access each cell of an array. Nevertheless, there is a way we can define two-dimensional arrays in Java—we can create a two-dimensional array as an array of arrays. That is, we can define a two-dimensional array to be an array with each of its cells being another array. Such a two-dimensional array is sometimes also called a *matrix*. In Java, we may declare a two-dimensional array as follows:

```
int[ ][ ] data = new int[8][10];
```

This statement creates a two-dimensional “array of arrays,” `data`, which is 8×10 , having 8 rows and 10 columns. That is, `data` is an array of length 8 such that each element of `data` is an array of length 10 of integers. (See Figure 3.7.) The following would then be valid uses of array `data` and `int` variables i , j , and k :

```
data[i][i+1] = data[i][i] + 3;
j = data.length;           // j is 8
k = data[4].length;        // k is 10
```

Two-dimensional arrays have many applications to numerical analysis. Rather than going into the details of such applications, however, we explore an application of two-dimensional arrays for implementing a simple positional game.

	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

Figure 3.7: Illustration of a two-dimensional integer array, `data`, which has 8 rows and 10 columns. The value of `data[3][5]` is 100 and the value of `data[6][2]` is 632.

Tic-Tac-Toe

As most school children know, *Tic-Tac-Toe* is a game played in a three-by-three board. Two players—X and O—alternate in placing their respective marks in the cells of this board, starting with player X. If either player succeeds in getting three of his or her marks in a row, column, or diagonal, then that player wins.

This is admittedly not a sophisticated positional game, and it's not even that much fun to play, since a good player O can always force a tie. Tic-Tac-Toe's saving grace is that it is a nice, simple example showing how two-dimensional arrays can be used for positional games. Software for more sophisticated positional games, such as checkers, chess, or the popular simulation games, are all based on the same approach we illustrate here for using a two-dimensional array for Tic-Tac-Toe.

The basic idea is to use a two-dimensional array, *board*, to maintain the game board. Cells in this array store values that indicate if that cell is empty or stores an X or O. That is, *board* is a three-by-three matrix, whose middle row consists of the cells `board[1][0]`, `board[1][1]`, and `board[1][2]`. In our case, we choose to make the cells in the *board* array be integers, with a 0 indicating an empty cell, a 1 indicating an X, and a -1 indicating an O. This encoding allows us to have a simple way of testing if a given board configuration is a win for X or O, namely, if the values of a row, column, or diagonal add up to 3 or -3 , respectively. We illustrate this approach in Figure 3.8.

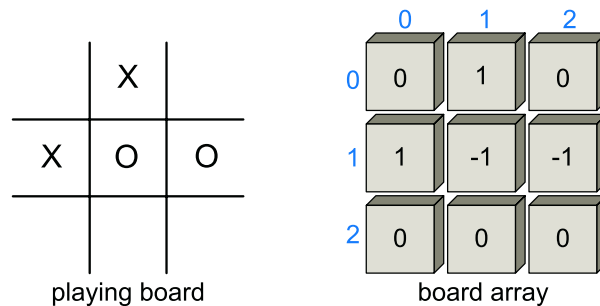


Figure 3.8: An illustration of a Tic-Tac-Toe board and the two-dimensional integer array, *board*, representing it.

We give a complete Java class for maintaining a Tic-Tac-Toe board for two players in Code Fragments 3.9 and 3.10. We show a sample output in Figure 3.9. Note that this code is just for maintaining the Tic-Tac-Toe board and registering moves; it doesn't perform any strategy or allow someone to play Tic-Tac-Toe against the computer. The details of such a program are beyond the scope of this chapter, but it might nonetheless make a good course project (see Exercise P-8.67).

```

1  /** Simulation of a Tic-Tac-Toe game (does not do strategy). */
2  public class TicTacToe {
3      public static final int X = 1, O = -1;           // players
4      public static final int EMPTY = 0;              // empty cell
5      private int board[ ][ ] = new int[3][3];         // game board
6      private int player;                             // current player
7      /** Constructor */
8      public TicTacToe() { clearBoard(); }
9      /** Clears the board */
10     public void clearBoard() {
11         for (int i = 0; i < 3; i++)
12             for (int j = 0; j < 3; j++)
13                 board[i][j] = EMPTY;                // every cell should be empty
14         player = X;                                   // the first player is 'X'
15     }
16     /** Puts an X or O mark at position i,j. */
17     public void putMark(int i, int j) throws IllegalArgumentException {
18         if ((i < 0) || (i > 2) || (j < 0) || (j > 2))
19             throw new IllegalArgumentException("Invalid board position");
20         if (board[i][j] != EMPTY)
21             throw new IllegalArgumentException("Board position occupied");
22         board[i][j] = player;                        // place the mark for the current player
23         player = - player;                           // switch players (uses fact that O = - X)
24     }
25     /** Checks whether the board configuration is a win for the given player. */
26     public boolean isWin(int mark) {
27         return ((board[0][0] + board[0][1] + board[0][2] == mark*3) // row 0
28             || (board[1][0] + board[1][1] + board[1][2] == mark*3) // row 1
29             || (board[2][0] + board[2][1] + board[2][2] == mark*3) // row 2
30             || (board[0][0] + board[1][0] + board[2][0] == mark*3) // column 0
31             || (board[0][1] + board[1][1] + board[2][1] == mark*3) // column 1
32             || (board[0][2] + board[1][2] + board[2][2] == mark*3) // column 2
33             || (board[0][0] + board[1][1] + board[2][2] == mark*3) // diagonal
34             || (board[2][0] + board[1][1] + board[0][2] == mark*3)); // rev diag
35     }
36     /** Returns the winning player's code, or 0 to indicate a tie (or unfinished game).*/
37     public int winner() {
38         if (isWin(X))
39             return(X);
40         else if (isWin(O))
41             return(O);
42         else
43             return(0);
44     }

```

Code Fragment 3.9: A simple, complete Java class for playing Tic-Tac-Toe between two players. (Continues in Code Fragment 3.10.)

```

45  /** Returns a simple character string showing the current board. */
46  public String toString() {
47      StringBuilder sb = new StringBuilder();
48      for (int i=0; i<3; i++) {
49          for (int j=0; j<3; j++) {
50              switch (board[i][j]) {
51                  case X:      sb.append("X"); break;
52                  case O:      sb.append("O"); break;
53                  case EMPTY:  sb.append(" "); break;
54              }
55              if (j < 2) sb.append(" | ");           // column boundary
56          }
57          if (i < 2) sb.append("\n-----\n");       // row boundary
58      }
59      return sb.toString();
60  }
61  /** Test run of a simple game */
62  public static void main(String[] args) {
63      TicTacToe game = new TicTacToe();
64      /* X moves: */           /* O moves: */
65      game.putMark(1,1);      game.putMark(0,2);
66      game.putMark(2,2);      game.putMark(0,0);
67      game.putMark(0,1);      game.putMark(2,1);
68      game.putMark(1,2);      game.putMark(1,0);
69      game.putMark(2,0);
70      System.out.println(game);
71      int winningPlayer = game.winner();
72      String[] outcome = {"O wins", "Tie", "X wins"}; // rely on ordering
73      System.out.println(outcome[1 + winningPlayer]);
74  }
75  }

```

Code Fragment 3.10: A simple, complete Java class for playing Tic-Tac-Toe between two players. (Continued from Code Fragment 3.9.)

```

O|X|O
-----
O|X|X
-----
X|O|X
Tie

```

Figure 3.9: Sample output of a Tic-Tac-Toe game.